



(21) 申请号 202410409399.2

G06N 20/00 (2019.01)

(22) 申请日 2024.04.07

G06F 17/18 (2006.01)

G06N 3/048 (2023.01)

(71) 申请人 中国核动力研究设计院

地址 610213 四川省成都市双流区长顺大道一段328号

(72) 发明人 刘东 陈俊辑 陈奇隆 彭航
潘俊杰 张斌 李庆 涂晓兰
陈长 肖聪 芦轲 李文杰
范佳锟 向迪 卢宗健 曾辉
崔显涛 赵晨 江勇 庞志鑫
邓平

(74) 专利代理机构 核工业专利中心 11007

专利代理师 胡维维

(51) Int. Cl.

G06F 17/13 (2006.01)

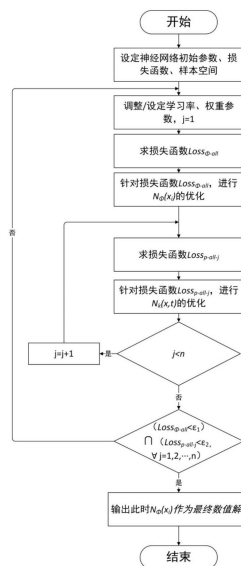
权利要求书2页 说明书14页 附图2页

(54) 发明名称

一种深度机器学习求解连续性积分方程的方法

(57) 摘要

本发明具体涉及一种深度机器学习求解连续性积分方程的方法,包括如下步骤:对包含未知函数和积分项的连续性积分方程中的积分项转化为被积函数的原函数表达式,将原积分方程转化为没有积分项的微分方程;给出被积函数原函数特定解的生成方法;对未知函数和被积函数原函数分别映射为神经网络函数,对未知函数和被积函数原函数的神经网络函数交替进行深度机器学习;完成深度机器学习的未知函数和被积函数原函数的神经网络函数分别为未知函数和被积函数原函数的数值解。本发明避免了积分项有限和代替积分项带来的系统性误差以及被积函数缺乏唯一性带来的收敛性问题,使其有效应用于中子输运理论、流体力学以及热力学等工业领域的核心方程求解。



1. 一种深度机器学习求解连续性积分方程的方法,其特征在于,包括如下步骤:

S1、对包含未知函数和积分项的连续性积分方程,使用牛顿-莱布尼茨公式将其中的积分项转化为被积函数的原函数表达式,将原积分方程转化为没有积分项的微分方程;

S2、给出被积函数原函数特定解的生成方法;

S3、对未知函数和被积函数原函数分别映射为神经网络函数,对未知函数和被积函数原函数的神经网络函数交替进行深度机器学习;

完成深度机器学习的未知函数和被积函数原函数的神经网络函数分别为未知函数和被积函数原函数的数值解。

2. 根据权利要求1所述的方法,其特征在于,S1中,连续性积分方程中的积分项为定积分项,使用牛顿-莱布尼茨公式将连续性积分方程中的定积分项转化为被积函数原函数在积分上下限处的定值相减。

3. 根据权利要求1所述的方法,其特征在于,S1中,连续性积分方程中的积分项为不定积分项,使用牛顿-莱布尼茨公式将连续性积分方程中的不定积分项转化为被积函数原函数在积分上下限对应变量处的函数值相减。

4. 根据权利要求1所述的方法,其特征在于,S1中,连续性积分方程中存在多个积分项时,使用牛顿-莱布尼茨公式将连续性积分方程中的各个积分项分别转化为各个被积函数原函数在积分上下限处的值相减;S3中,对未知函数和各个被积函数原函数分别映射为神经网络函数,对未知函数和各个被积函数原函数的神经网络函数交替进行深度机器学习;完成深度机器学习的未知函数和各个被积函数原函数的神经网络函数分别为未知函数和各个被积函数原函数的数值解。

5. 根据权利要求1所述的方法,其特征在于,S2中,建立被积函数原函数的定解约束,使得被积函数原函数在积分上限或下限处的函数值为常数。

6. 根据权利要求5所述的方法,其特征在于,S2中,令被积函数原函数在积分下限或上限处的常数值为0,使得由积分上下限表达的被积函数原函数等于定积分项。

7. 根据权利要求1所述的方法,其特征在于,S3,包括如下步骤:

基于物理信息指引的神经网络模型或其他神经网络模型,分别建立未知函数和被积函数原函数的神经网络函数;

分别建立未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数;

对未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别设定收敛准则;

对未知函数神经网络函数和被积函数原函数神经网络函数交替迭代进行深度机器学习;

当未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别满足收敛准则后,完成深度机器学习;

完成深度机器学习的未知函数神经网络函数和被积函数原函数神经网络函数分别为未知函数和被积函数原函数的数值解。

8. 根据权利要求7所述的方法,其特征在于,未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别由控制方程损失函数、未知函数定解约束损失函

数、被积函数原函数定解约束损失函数和边界条件损失函数设置不同的权重相加得到；其中，分别将未知函数神经网络函数和被积函数神经网络函数作为未知函数和被积函数原函数的试函数，带入微分方程得到控制方程损失函数；根据被积函数原函数定解约束建立被积函数原函数定解约束损失函数；根据未知函数定解约束建立未知函数定解损失函数。

9. 根据权利要求8所述的方法，其特征在于，对未知函数神经网络函数和被积函数原函数神经网络函数交替迭代进行深度机器学习过程中，分别不断调整未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数中各损失函数的权重值，直至未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别满足收敛准则。

10. 根据权利要求9所述的方法，其特征在于，未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别设定的收敛准则为：未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别小于等于设定值 ε 。

一种深度机器学习求解连续性积分方程的方法

技术领域

[0001] 本发明涉及数值计算技术领域,特别是涉及一种深度机器学习求解连续性积分方程的方法。

背景技术

[0002] 在中子输运理论、流体力学、热力学、电磁学等众多的工业领域,积分方程广泛存在,其数值求解方法一直是计算数学领域的难点与热点。一般来说,具有实际应用价值的积分方程往往只能通过数值方法得到近似数值解,经典方法包括有限差分逼近法、逐次逼近法、待定系数法等。

[0003] 近年来,随着人工智能技术的发展,通过深度学习求解微分方程的方法已经获得了重要的进展,同时,采用该方法求解积分方程,学术上也进行了积极的探索。

[0004] 已经有学者给出了多维Fredholm方程以及体积积分方程的深度学习方法,也有学者在边界积分方程上进行了尝试。在这些方法中,积分方程积分项的深度学习损失函数构造,均采用了将积分项转换为有限项和的方式。

[0005] 不同的数值积分公式,其积分系数是不同的,其选取的方法会直接影响积分的精度与收敛性,常用的方法如Gauss求积公式。数值实验表明这种方式是可行的,但计算精度并不高,其与解析解标准差在 10^{-2} 到 10^{-3} 之间。造成这种情况的重要原因在于通过积分项有限和代替积分项的方法带来了系统误差,同时由于相同积分值的被积函数缺乏唯一性,基于未知函数构造的积分方程损失函数,对深度学习过程收敛具有很大的挑战。

[0006] 要想将深度学习技术应用于前述中子输运理论、流体力学以及热力学等工业领域的核心方程求解,需要克服使用有限项和代替积分项的系统误差,还要解决深度学习过程中由于被积函数缺乏唯一性带来的收敛问题。

发明内容

[0007] 本发明的目的在于,提供一种深度机器学习求解连续性积分方程的方法,解决现有深度学习求解连续性积分方程的方法中通过积分项有限和代替积分项带来的系统性误差以及被积函数缺乏唯一性带来的收敛性问题,导致其无法有效应用于中子输运理论、流体力学以及热力学等工业领域的核心方程求解的问题,避免积分项有限和代替积分项带来的系统性误差以及被积函数缺乏唯一性带来的收敛性问题,使其有效应用于中子输运理论、流体力学以及热力学等工业领域的核心方程求解。

[0008] 为了实现上述目的,本发明提供一种深度机器学习求解连续性积分方程的方法,包括如下步骤:

[0009] S1、对包含未知函数和积分项的连续性积分方程,使用牛顿-莱布尼茨公式将其中的积分项转化为被积函数的原函数表达式,将原积分方程转化为没有积分项的微分方程;

[0010] S2、给出被积函数原函数特定解的生成方法;

[0011] S3、对未知函数和被积函数原函数分别映射为神经网络函数,对未知函数和被积

函数原函数的神经网络函数交替进行深度机器学习；

[0012] 完成深度机器学习的未知函数和被积函数原函数的神经网络函数分别为未知函数和被积函数原函数的数值解。

[0013] 作为其中一种可能的实现方式,S1中,连续性积分方程中的积分项为定积分项,使用牛顿-莱布尼茨公式将连续性积分方程中的定积分项转化为被积函数原函数在积分上下限处的定值相减。

[0014] 作为其中一种可能的实现方式,S1中,连续性积分方程中的积分项为不定积分项,使用牛顿-莱布尼茨公式将连续性积分方程中的不定积分项转化为被积函数原函数在积分上下限对应变量处的函数值相减。

[0015] 作为其中一种可能的实现方式,S1中,连续性积分方程中存在多个积分项时,使用牛顿-莱布尼茨公式将连续性积分方程中的各个积分项分别转化为各个被积函数原函数在积分上下限处的值相减；

[0016] S3中,对未知函数和各个被积函数原函数分别映射为神经网络函数,对未知函数和各个被积函数原函数的神经网络函数交替进行深度机器学习；

[0017] 完成深度机器学习的未知函数和各个被积函数原函数的神经网络函数分别为未知函数和各个被积函数原函数的数值解。

[0018] 作为其中一种可能的实现方式,S2中,建立被积函数原函数的定解约束,使得被积函数原函数在积分上限或下限处的函数值为常数。

[0019] 作为其中一种可能的实现方式,S2中,令被积函数原函数在积分下限或上限处的常数值为0,使得由积分上下限表达的被积函数原函数等于定积分项。

[0020] 作为其中一种可能的实现方式,S3,包括如下步骤：

[0021] 基于物理信息指引的神经网络模型或其他神经网络模型,分别建立未知函数和被积函数原函数的神经网络函数；

[0022] 分别建立未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数；

[0023] 对未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别设定收敛准则；

[0024] 对未知函数神经网络函数和被积函数原函数神经网络函数交替迭代进行深度机器学习；

[0025] 当未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别满足收敛准则后,完成深度机器学习；

[0026] 完成深度机器学习的未知函数神经网络函数和被积函数原函数神经网络函数分别为未知函数和被积函数原函数的数值解。

[0027] 作为其中一种可能的实现方式,未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别由控制方程损失函数、未知函数定解约束损失函数、被积函数原函数定解约束损失函数和边界条件损失函数设置不同的权重相加得到；

[0028] 其中,分别将未知函数神经网络函数和被积函数神经网络函数作为未知函数和被积函数原函数的试函数,带入微分方程得到控制方程损失函数；

[0029] 根据被积函数原函数定解约束建立被积函数原函数定解约束损失函数；

[0030] 根据未知函数定解约束建立未知函数定解损失函数。

[0031] 作为其中一种可能的实现方式,对未知函数神经网络函数和被积函数原函数神经网络函数交替迭代进行深度机器学习过程中,分别不断调整未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数中各损失函数的权重值,减少陷入局部最优的可能性,直至未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别满足收敛准则。

[0032] 作为其中一种可能的实现方式,未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别设定的收敛准则为:未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别小于等于设定值 ε 。

[0033] 本发明的有益技术效果:

[0034] 本发明的深度机器学习求解连续性积分方程的方法,对包含未知函数和积分项的连续性积分方程的积分项转化为被积函数的原函数表达式,将原积分方程转化为无积分项的微分方程;给出被积函数原函数特定解的生成方法;对未知函数和被积函数原函数分别映射为神经网络函数,对未知函数和被积函数原函数的神经网络函数交替进行深度机器学习;完成深度机器学习的未知函数和被积函数原函数的神经网络函数分别为未知函数和被积函数原函数的数值解;从而克服现有深度学习求解连续性积分方程方法中有限项和代替积分项带来的系统误差以及以及被积函数缺乏唯一性带来的收敛性问题,使其有效应用于中子输运理论、流体力学以及热力学等工业领域的核心方程求解。

附图说明

[0035] 图1为本发明的深度机器学习求解连续性积分方程的方法的一个实施例的流程图;

[0036] 图2为实施例2中未知函数 $\varphi(x)$ 的数值解与解析解散点分布图;

[0037] 图3为实施例2中原函数 $F_0(x, t)$ 的数值解散点分布图;

[0038] 图4为实施例3中未知函数 $\varphi(x)$ 的数值解与解析解散点分布图;

[0039] 图5为实施例3中原函数 $F_0(x, t)$ 的数值解散点分布图。

具体实施方式

[0040] 除非另有定义,本文所使用的所有的技术和科学术语与属于本申请的技术领域的技术人员通常理解的含义相同;本文在申请的说明书中所使用的术语只是为了描述具体的实施例的目的,不是旨在于限制本申请;本申请的说明书和权利要求书及上述附图说明中的术语“包括”和“设有”以及它们的任何变形,意图在于覆盖不排他的包含。

[0041] 在本文中提及“实施例”意味着,结合实施例描述的特定特征、结构或特性可以包含在本申请的至少一个实施例中。在说明书中的各个位置出现该短语并不一定均是指相同的实施例,也不是与其它实施例互斥的独立的或备选的实施例。本领域技术人员显式地和隐式地理解的是,本文所描述的实施例可以与其它实施例相结合。

[0042] 下面结合附图和具体实施例对本发明的技术方案进行清楚、完整地描述。

[0043] 参见图1,本发明提供一种深度机器学习求解连续性积分方程的方法,包括以下步骤:

[0044] S1、对包含未知函数和积分项的连续性积分方程,使用牛顿-莱布尼茨公式将其中的积分项转化为被积函数的原函数表达式,将原积分方程转化为没有积分项的微分方程;

[0045] S2、给出被积函数原函数特定解的生成方法;

[0046] S3、对未知函数和被积函数原函数分别映射为神经网络函数,对未知函数和被积函数原函数的神经网络函数交替进行深度机器学习;

[0047] 完成深度机器学习的未知函数和被积函数原函数的神经网络函数分别为未知函数和被积函数原函数的数值解。

[0048] 作为其中一种可能的实现方式,S1中,连续性积分方程中的积分项为定积分项,使用牛顿-莱布尼茨公式将连续性积分方程中的定积分项转化为被积函数原函数在积分上下限处的定值相减。

[0049] 作为其中一种可能的实现方式,S1中,连续性积分方程中的积分项为不定积分项,使用牛顿-莱布尼茨公式将连续性积分方程中的不定积分项转化为被积函数原函数在积分上下限对应变量处的函数值相减。

[0050] 作为其中一种可能的实现方式,S1中,连续性积分方程中存在多个积分项时,使用牛顿-莱布尼茨公式将连续性积分方程中的各个积分项分别转化为各个被积函数原函数在积分上下限处的值相减;

[0051] S3中,对未知函数和各个被积函数原函数分别映射为神经网络函数,对未知函数和各个被积函数原函数的神经网络函数交替进行深度机器学习;

[0052] 完成深度机器学习的未知函数和各个被积函数原函数的神经网络函数分别为未知函数和各个被积函数原函数的数值解。

[0053] 作为其中一种可能的实现方式,S2中,建立被积函数原函数的定解约束,使得被积函数原函数在积分上限或下限处的函数值为常数。

[0054] 作为其中一种可能的实现方式,S2中,令被积函数原函数在积分下限或上限处的常数值为0,使得由积分上下限表达的被积函数原函数等于定积分项。

[0055] 作为其中一种可能的实现方式,S3,包括如下步骤:

[0056] 基于物理信息指引的神经网络模型或其他神经网络模型,分别建立未知函数和被积函数原函数的神经网络函数;

[0057] 分别建立未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数;

[0058] 对未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别设定收敛准则;

[0059] 对未知函数神经网络函数和被积函数原函数神经网络函数交替迭代进行深度机器学习;

[0060] 当未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别满足收敛准则后,完成深度机器学习;

[0061] 完成深度机器学习的未知函数神经网络函数和被积函数原函数神经网络函数分别为未知函数和被积函数原函数的数值解。

[0062] 作为其中一种可能的实现方式,未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别由控制方程损失函数、未知函数定解约束损失函数、被

积函数原函数定解约束损失函数和边界条件损失函数设置不同的权重相加得到;其中,分别将未知函数神经网络函数和被积函数神经网络函数作为未知函数和被积函数原函数的试函数,带入微分方程得到控制方程损失函数;根据被积函数原函数定解约束建立被积函数原函数定解约束损失函数;根据未知函数定解约束建立未知函数定解损失函数。

[0063] 作为其中一种可能的实现方式,对未知函数神经网络函数和被积函数原函数神经网络函数交替迭代进行深度机器学习过程中,分别不断调整未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数中各损失函数的权重值,减少陷入局部最优的可能性,直至未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别满足收敛准则。

[0064] 作为其中一种可能的实现方式,未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别设定的收敛准则为:未知函数神经网络函数的损失函数和被积函数原函数神经网络函数的损失函数分别小于等于设定值 ε 。

[0065] 下面结合附图和实施例对本发明做进一步详细地说明。

[0066] 实施例1

[0067] 本实施例提供一种深度机器学习求解连续性积分方程的方法,包括以下步骤:

[0068] S1、对包含未知函数和积分项的连续性积分方程,使用牛顿-莱布尼茨公式将其中的积分项转化为被积函数的原函数表达式,将原积分方程转化为没有积分项的微分方程;

[0069] S2、给出被积函数原函数特定解的生成方法;

[0070] S3、对未知函数和被积函数原函数分别映射为神经网络函数,对未知函数和被积函数原函数的神经网络函数交替进行深度机器学习;

[0071] 完成深度机器学习的未知函数和被积函数原函数的神经网络函数分别为未知函数和被积函数原函数的数值解。

[0072] 积分方程的一般形式为:

$$[0073] \quad a(x) \phi(x) = \lambda \int_a^x K(x, t) M[\phi(t)] dt + f(x)$$

[0074] 其中, λ 为参数, $a(x)$ 、 $f(x)$ 、积分核 $K(x, t)$ 为已知函数, $\phi(x)$ 为未知函数, $M[\phi(t)]$ 为 $\phi(t)$ 的已知泛函, x 为积分方程自变量, t 为积分方程积分符号下的自变量;当 $M[\phi(t)]$ 为 $\phi(t)$ 的线性泛函时,积分方程为线性积分方程, $M[\phi(t)]$ 可直接用 $\phi(t)$ 表示,此时,积分上下限为常数时积分方程为Fredholm方程,积分上下限含有变量时积分方程为Volterra方程;考虑到积分上下限为常数的Fredholm方程可以看成是积分上下限含有变量的Volterra方程的特殊形式,同时,一维方程的求解方法通常可以扩展到高维方程,下面以深度机器学习求解一维Volterra方程为例进行说明。

[0075] 在本实施例中,S1中,连续性积分方程为一维Volterra方程,其表达式为:

$$[0076] \quad a(x) \phi(x) = \lambda \int_a^x K(x, t) M[\phi(t)] dt + f(x) \quad (1)$$

[0077] 其中, λ 为参数, $a(x)$ 、 $f(x)$ 、积分核 $K(x, t)$ 为已知函数, $\phi(x)$ 为未知函数, $M[\phi(t)]$ 为 $\phi(t)$ 的线性泛函, x 为积分方程自变量, t 为积分方程积分符号下的自变量;

[0078] 假设方程(1)中被积函数 $K(x, t)M[\phi(t)]$ 是关于自变量 t 的连续函数,根据微积分学基本原理,存在关于自变量 t 的被积函数 $K(x, t)M[\phi(t)]$ 的原函数 $F(x, t)$,满足:

[0079] $F(x, t)' = K(x, t)M[\varphi(t)]$ (2)

[0080] 根据牛顿-莱布尼兹公式有:

$$[0081] \int_a^x K(x, t)M[\phi(t)] dt = F(x, x) - F(x, a) \quad (3)$$

[0082] 将方程 (3) 代入方程 (1) 得到:

$$[0083] a(x) \varphi(x) = \lambda(F(x, x) - F(x, a)) + f(x) \quad (4)$$

[0084] 从而将给定的积分方程转换为没有积分项的微分方程。

[0085] 根据微积分性质,同一被积函数的原函数为一簇函数,满足方程 (4) 的原函数是不确定的;同时,深度学习求解微分方程过程中,不确定解的方程很难收敛,或者收敛速度会受到大幅度影响,因此需要确定方程 (4) 的一个原函数方便后续进行深度学习。

[0086] 在本实施例中,S2中,方程 (2) 的原函数为一组相差实数C的函数,称为原函数簇 $F_n(x, t)$;设 $F_1(x, t)$ 为原函数簇 $F_n(x, t)$ 中一个特定的原函数,则原函数簇 $F_n(x, t)$ 满足以下形式: $F_n(x, t) = F_1(x, t) + C_n$,其中, C_n 为任意的实数值, $n=0, 1, 2, \dots$;

[0087] 令 $C_n = -F_1(x, a)$,则有原函数簇 $F_n(x, t)$ 中确定的原函数 $F_0(x, t)$ 为:

$$[0088] F_0(x, t) = F_1(x, t) - F_1(x, a) \quad (5)$$

[0089] 当 $t=a$ 时,有 $F_0(x, a) = 0$,此时为原函数簇 $F_n(x, t)$ 在满足一定的约束条件下,找到了一个确定的原函数 $F_0(x, t)$,将方程 (5) 带入方程 (4) 得到:

$$[0090] a(x) \varphi(x) = \lambda F_0(x, x) + f(x) \quad (6)$$

[0091] 此时,确定被积函数原函数特定解的约束条件为: $t=a, F_0(x, a) = 0$ 。

[0092] 在本实施例中,S3中,将未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 分别使用神经网络函数进行映射,同时分别构建相应的损失函数,通过交替迭代深度机器学习,收敛得到的未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的神经网络函数分别为未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的数值解。

[0093] 作为其中一种可能的实现方式,S3,包括如下步骤:

[0094] S30、基于物理信息指引的神经网络模型或其他神经网络模型,分别建立未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的神经网络函数;

[0095] S31、分别将未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的神经网络函数作为未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的试函数,带入微分方程得到控制方程损失函数,根据原函数 $F_0(x, t)$ 定解约束建立原函数定解约束损失函数,根据未知函数 $\varphi(x)$ 定解约束建立未知函数定解损失函数,建立边界条件损失函数;

[0096] S32、根据S31中的控制方程损失函数、原函数定解约束损失函数、边界条件损失函数以及未知函数定解损失函数建立统一损失函数;

[0097] S33、根据S32中的统一损失函数,对未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的神经网络函数交替进行深度机器学习;

[0098] 完成深度机器学习的未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的神经网络函数分别为未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的数值解。

[0099] 作为其中一种可能的实现方式,S30中,基于物理信息指引的神经网络模型,对未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 分别建立采用全连接的神经网络函数 $N(\vec{x})$:

$$[0100] N(\vec{x}) = f(\vec{x}, w, b, l, n) \quad (7)$$

[0101] 其中, $\vec{x}$ 为神经网络输入向量, $N(\vec{x})$ 为神经网络输出, w 为神经网络连接权重; b 为神经网络偏置项, l 为网络层数, n 为隐藏神经单元个数。

[0102] 需要说明的是, 可以根据深度学习的需要, 选择任意类型的神经网络函数来求解目标方程, 本发明对神经网络函数的类型不做限定。

[0103] 作为其中一种可能的实现方式, S31 中, 对于控制方程损失函数, 令 $N_\phi(\vec{x}) = \phi(x)$, $N_p(\vec{x}, \vec{t}) = F_0(x, t)$, 其中, $N_\phi(\vec{x})$ 为未知函数 $\phi(x)$ 的神经网络函数, $N_p(\vec{x}, \vec{t})$ 为原函数 $F_0(x, t)$ 的神经网络函数, $\vec{x}$ 表示积分方程自变量 x , $\vec{t}$ 表示积分方程积分符号下的自变量 t ; 如果未知函数 $\phi(x)$ 和原函数 $F_0(x, t)$ 分别是高维方程, $\vec{x}$ 和 $\vec{t}$ 分别根据 x 和 t 的维度做适应性调整;

[0104] 用 $N_p'(\vec{x}, \vec{t})$ 表示 $N_p(\vec{x}, \vec{t})$ 关于 $\vec{t}$ 的导数/偏导数, 即: $N_p'(\vec{x}, \vec{t}) = F_0'(x, t)$;

[0105] 将 $N_p'(\vec{x}, \vec{t})$ 和 $F_0'(x, t)$ 的等价关系代入方程 (6), 得到控制方程残差表示的控制方程损失函数 $Loss_\phi$ 为:

$$[0106] \quad Loss_\phi = \frac{1}{|I|} \sum_i [a(\vec{x}_i) N_\phi(\vec{x}_i) - \lambda N_p(\vec{x}_i, \vec{x}_i) - f(\vec{x}_i)]^2 \quad (8)$$

[0107] 将 $N_p'(\vec{x}, \vec{t})$ 和 $F_0'(x, t)$ 的等价关系代入方程 (2), 得到原函数 $F_0(x, t)$ 的导数/偏导数与未知函数 $\phi(x)$ 形成的由残差表示的控制方程损失函数 $Loss_p$ 为:

$$[0108] \quad Loss_p = \frac{1}{|I|} \sum_i [N_p'(\vec{x}_i, \vec{t}_i) - K(\vec{x}_i, \vec{t}_i) M[N_\phi(\vec{t}_i)]]^2 \quad (9)$$

[0109] 其中, i 为机器学习需要的每个样本点, 可按照一定的概率密度分布生成, 或者根据方程的形式采用不同密度分布策略生成, I 为机器学习需要的样本点总数。

[0110] 作为其中一种可能的实现方式, S31 中, 对于原函数 $F_0(x, t)$ 的定解约束损失函数, 令 $N(\vec{x}, a) = F_0(x, a)$, 则得到神经网络函数表示的原函数定解约束损失函数 $Loss_d$ 为:

$$[0111] \quad Loss_d = \frac{1}{|M|} \sum_m [N_p(\vec{x}_m, a) - F_0(\vec{x}_m, a)]^2 \quad (10)$$

[0112] 其中, m 为原函数定解约束项取值域 $\{\vec{x}\}$ 上的每个机器学习样本点, M 为原函数定解约束项取值域 $\{\vec{x}\}$ 上总的机器学习样本点数目。

[0113] 对于未知函数 $\phi(x)$ 的定解约束损失函数, 方程 (9) 中的泛函 $M[N_\phi(t)]$ 可能存在某些非线性特殊形式, 使得 $M[N_\phi(t)]$ 与 $N_\phi(t)$ 不是一一映射, 即可能存在多种 $N_\phi(t)$ 映射为同一个 $M[N_\phi(t)]$, 这意味着神经网络函数 $N_\phi(\vec{x})$ 的深度机器学习结果可能有多个解, 神经网络函数 $N_\phi(\vec{x})$ 的收敛不唯一, 这样的特性通常会导致神经网络函数 $N_\phi(\vec{x})$ 的收敛效率降低或者难以收敛。例如, 当 $M[N_\phi(t)]$ 为非线性函数, $M[N_\phi(t)] = N_\phi(t)^2$, 方程 (9) 的非零解并不唯一, $\pm N_\phi(t)$ 均为方程 (9) 的非零解。

[0114] 为了加快神经网络函数 $N_\phi(\vec{x})$ 的收敛, 作为其中一种可能的实现方式, S31 中, 引

入 $N_{\phi}(t) > 0$ 或 $N_{\phi}(t) < 0$;

[0115] 对于 $N_{\phi}(t) > 0$,设定未知函数定解约束损失函数 $Loss_c$ 为:

$$[0116] \quad Loss_c = \frac{1}{|P|} \sum_p \left[N_{\phi}(\vec{t}_p) - abs(N_{\phi}(\vec{t}_p)) \right]^2 \quad (11)$$

[0117] 对于 $N_{\phi}(t) < 0$,设定未知函数定解约束损失函数 $Loss_c$ 为:

$$[0118] \quad Loss_c = \frac{1}{|P|} \sum_p \left[N_{\phi}(\vec{t}_p) + abs(N_{\phi}(\vec{t}_p)) \right]^2 \quad (12)$$

[0119] 其中, p 为未知函数定解约束项取值域上的每个机器学习样本点, P 为未知函数定解约束项取值域上总的机器学习样本点数目, $abs()$ 为取绝对值函数。

[0120] 针对实际问题,应根据 $M[\phi(t)]$ 的形式确定相应的定解损失函数,确保残差函数的唯一性,从而加速神经网络函数 $N_{\phi}(\vec{x})$ 的收敛。

[0121] 对于边界条件损失函数,需要根据具体问题进行设置。作为其中一种可能的实现方式,S31中,设 $\vec{x}_j$ 为神经网络函数 $N_{\phi}(\vec{x})$ 的边界域上的机器学习样本点,用 $\phi_b(\vec{x}_j)$ 表示未知函数 $\phi(x)$ 的边界条件函数, $\phi_b(\vec{x}_j)$ 的取值根据不同的边界条件确定,得到边界条件损失函数 $Loss_b$ 为:

$$[0122] \quad Loss_b = \frac{1}{|J|} \sum_j \left[N_{\phi}(\vec{x}_j) - \phi_b(\vec{x}_j) \right]^2 \quad (13)$$

[0123] 其中, j 为神经网络函数 $N_{\phi}(\vec{x})$ 边界条件取值域上的每个机器学习样本点, J 为神经网络函数 $N_{\phi}(\vec{x})$ 边界条件取值域上总的机器学习样本点数目。

[0124] 作为其中一种可能的实现方式,S32中,将S31中的损失函数加权,得到神经网络函数 $N_{\phi}(\vec{x})$ 的损失函数 $Loss_{\phi-all}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $Loss_{p-all}$ 为:

$$[0125] \quad Loss_{\phi-all} = P_{\phi} Loss_{\phi} + P_c Loss_c + P_b Loss_b \quad (14)$$

$$[0126] \quad Loss_{p-all} = P_p Loss_p + P_d Loss_d \quad (15)$$

[0127] 其中, P_{ϕ} 为控制方程损失函数 $Loss_{\phi}$ 的权重, P_p 为控制方程损失函数 $Loss$ 的权重, P_b 为边界条件损失函数 $Loss_b$ 的权重, P_c 为未知函数定解约束损失函数 $Loss_c$ 的权重, P_d 为原函数定解约束损失函数 $Loss_d$ 的权重。

[0128] 作为其中一种可能的实现方式,S33中,神经网络函数 $N_{\phi}(\vec{x})$ 的损失函数 $Loss_{\phi-all}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $Loss_{p-all}$ 分别设定收敛准则;

[0129] 对神经网络函数 $N_{\phi}(\vec{x})$ 和 $N_p(\vec{x}, \vec{t})$ 交替进行深度机器学习,为根据方程(14)和(15)确定的加权损失函数,分别交替调整神经网络函数 $N_{\phi}(\vec{x})$ 和 $N_p(\vec{x}, \vec{t})$ 中的 w 和 b ,使得函数神经网络函数 $N_{\phi}(\vec{x})$ 的损失函数 $Loss_{\phi-all}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $Loss_{p-all}$ 分别满足收敛准则;

[0130] 当神经网络函数 $N_{\phi}(\vec{x})$ 的损失函数 $Loss_{\phi-all}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函

数 $\text{Loss}_{p\text{-all}}$ 分别满足收敛准则后,完成深度机器学习;

[0131] 完成深度机器学习的神经网络函数 $N_\phi(\vec{x})$ 和 $N_p(\vec{x}, \vec{t})$ 分别为未知函数 $\phi(x)$ 和原函数 $F_0(x, t)$ 的数值解。

[0132] 作为其中一种可能的实现方式,S33中,神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 分别设定的收敛准则为:神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 分别小于等于设定值 ϵ 。

[0133] 实施例2

[0134] 本实施例提供一种深度机器学习求解连续性积分方程的方法,包括以下步骤:

[0135] S1、对包含未知函数和积分项的连续性积分方程,使用牛顿-莱布尼茨公式将其中的积分项转化为被积函数的原函数表达式,将原积分方程转化为没有积分项的微分方程;

[0136] S2、给出被积函数原函数特定解的生成方法;

[0137] S3、对未知函数和被积函数原函数分别映射为神经网络函数,对未知函数和被积函数原函数的神经网络函数交替进行深度机器学习;

[0138] 完成深度机器学习的未知函数和被积函数原函数的神经网络函数分别为未知函数和被积函数原函数的数值解。

[0139] 在本实施例中,S1中,连续性积分方程为第二类线性Fredholm方程,其表达式为:

$$[0140] \quad \phi(x) + \int_0^1 x e^t \phi(t) dt = e^{-x} \quad (16)$$

[0141] 其中, x 为积分方程自变量, t 为积分方程积分符号下的自变量, $\phi(x)$ 为未知函数;

[0142] 假设方程(16)中被积函数 $x e^t \phi(t)$ 是关于自变量 t 的连续函数,根据微积分学基本原理,存在被积函数 $x e^t \phi(t)$ 的原函数 $F(x, t)$,满足:

$$[0143] \quad F(x, t)' = x e^t \phi(t) \quad (17)$$

[0144] 根据牛顿-莱布尼兹公式有:

$$[0145] \quad \int_0^1 x e^t \phi(t) dt = F(x, x) - F(x, 0) \quad (18)$$

[0146] 将方程(18)代入方程(16)得到:

$$[0147] \quad \phi(x) + (F(x, x) - F(x, 0)) = e^{-x} \quad (19)$$

[0148] 从而将原积分方程转换为没有积分项的微分方程。

[0149] 在本实施例中,S2中,方程(17)的原函数为一组相差实数 C 的函数,称为原函数簇 $F_n(x, t)$;设 $F_1(x, t)$ 为原函数簇 $F_n(x, t)$ 中一个特定的原函数,则原函数簇 $F_n(x, t)$ 满足以下形式: $F_n(x, t) = F_1(x, t) + C_n$,其中, C_n 为任意的实数值, $n=0, 1, 2, \dots$;

[0150] 令 $C_n = -F_1(x, 0)$,则有原函数簇 $F_n(x, t)$ 中确定的原函数 $F_0(x, t)$ 为:

$$[0151] \quad F_0(x, t) = F_1(x, t) - F_1(x, 0) \quad (20)$$

[0152] 当 $t=0$ 时,有 $F_0(x, 0) = 0$,此时为原函数簇 $F_n(x, t)$ 在满足一定的约束条件下,找到了一个确定的原函数 $F_0(x, t)$,将方程(20)带入方程(19)得到:

$$[0153] \quad \phi(x) + F_0(x, x) = e^{-x} \quad (21)$$

[0154] 此时,确定被积函数原函数特定解的约束条件为: $t=0, F_0(x, 0) = 0$ 。

[0155] 在本实施例中,S3中,将未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 分别使用神经网络函数进行映射,同时分别构建相应的损失函数,通过交替迭代深度机器学习,收敛得到的未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的神经网络函数分别为未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的数值解。

[0156] 作为其中一种可能的实现方式,S3,包括如下步骤:

[0157] S30、基于物理信息指引的神经网络模型或其他神经网络模型,分别建立未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的神经网络函数;

[0158] S31、分别将未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的神经网络函数作为未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的试函数,带入微分方程得到控制方程损失函数;

[0159] S32、根据S31中的控制方程损失函数建立统一损失函数;

[0160] S33、根据S32中的统一损失函数,对未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的神经网络函数交替进行深度机器学习;

[0161] 完成深度机器学习的未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的神经网络函数分别为未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的数值解。

[0162] 作为其中一种可能的实现方式,S30中,基于物理信息指引的神经网络模型,对未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 分别建立采用基本的全连接的神经网络函数 $N(\vec{x})$:

$$[0163] \quad N(\vec{x}) = f(\vec{x}, w, b, l, n) \quad (22)$$

[0164] 其中, $\vec{x}$ 为神经网络输入向量, $N(\vec{x})$ 为神经网络输出, w 为神经网络连接权重; b 为神经网络偏置项; l 为网络层数, $l=9$; n 为隐藏神经单元个数, $n=40$ 。

[0165] 需要说明的是,可以根据深度学习的需要,选择任意类型的神经网络函数来求解目标方程,本发明对神经网络函数的类型不做限定。

[0166] 作为其中一种可能的实现方式, S31中,对于控制方程损失函数,令 $N_\varphi(\vec{x}) = \varphi(x)$, $N_p(\vec{x}, \vec{t}) = F_0(x, t)$, 其中, $N_\varphi(\vec{x})$ 为未知函数 $\varphi(x)$ 的神经网络函数, $N_p(\vec{x}, \vec{t})$ 为原函数 $F_0(x, t)$ 的神经网络函数, $\vec{x}$ 表示积分方程自变量 x , $\vec{t}$ 表示积分方程积分符号下的自变量 t ;

[0167] 用 $N_p'(\vec{x}, \vec{t})$ 表示 $N_p(\vec{x}, \vec{t})$ 关于 $\vec{t}$ 的导数/偏导数, 即: $N_p'(\vec{x}, \vec{t}) = F_0'(x, t)$;

[0168] 将 $N_p'(\vec{x}, \vec{t})$ 和 $F_0'(x, t)$ 的等价关系代入方程 (21), 得到控制方程残差表示的控制方程损失函数 $Loss_\varphi$ 为:

$$[0169] \quad Loss_\varphi = \frac{1}{|I|} \sum_i [a(\vec{x}_i) N_\varphi(\vec{x}_i) - \lambda N_p(\vec{x}_i, \vec{x}_i) - f(\vec{x}_i)]^2 \quad (23)$$

[0170] 将 $N_p'(\vec{x}, \vec{t})$ 和 $F_0'(x, t)$ 的等价关系代入方程 (17), 得到原函数 $F_0(x, t)$ 的导数/偏导数与未知函数 $\varphi(x)$ 形成的由残差表示的控制方程损失函数 $Loss_p$ 为:

$$[0171] \quad Loss_p = \frac{1}{|I|} \sum_i [N_p'(\vec{x}_i, \vec{t}_i) - K(\vec{x}_i, \vec{t}_i) M[N_\varphi(\vec{t}_i)]]^2 \quad (24)$$

[0172] 其中, i 为机器学习需要的每个样本点, 采用随机分布生成; I 为机器学习需要的样

本点总数, $I=500$ 。

[0173] 作为其中一种可能的实现方式, S32中, 将S31中的损失函数加权, 得到神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 为:

$$[0174] \quad \text{Loss}_{\phi\text{-all}} = P_\phi \text{Loss}_\phi \quad (25)$$

$$[0175] \quad \text{Loss}_{p\text{-all}} = P_p \text{Loss}_p \quad (26)$$

[0176] 其中, P_ϕ 为控制方程损失函数 Loss_ϕ 的权重, $P_\phi=1$; P_p 为控制方程损失函数 Loss_p 的权重, $P_p=1$ 。

[0177] 作为其中一种可能的实现方式, S33中, 神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 分别设定收敛准则;

[0178] 对神经网络函数 $N_\phi(\vec{x})$ 和 $N_p(\vec{x}, \vec{t})$ 交替进行深度机器学习, 为根据方程 (25) 和 (26) 确定的加权损失函数, 分别交替调整神经网络函数 $N_\phi(\vec{x})$ 和 $N_p(\vec{x}, \vec{t})$ 中的 w 和 b , 使得函数神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 分别满足收敛准则;

[0179] 当神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 分别满足收敛准则后, 完成深度机器学习;

[0180] 完成深度机器学习的神经网络函数 $N_\phi(\vec{x})$ 和 $N_p(\vec{x}, \vec{t})$ 分别为未知函数 $\phi(x)$ 和原函数 $F_0(x, t)$ 的数值解;

[0181] 深度机器学习采用ADAM机器学习算法, 激活函数选用tanh, 神经网络初始值采用高斯随机分布, 机器学习率从 $1e^{-4}$ 开始, 随着学习次数增加, 逐渐下降, 训练到神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 均不再下降。

[0182] 作为其中一种可能的实现方式, S33中, 神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 分别设定的收敛准则为: 神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 分别小于等于设定值 ϵ 。

[0183] 未知函数 $\phi(x)$ 的解析解为:

$$[0184] \quad \phi(x) = e^{-x} - \frac{x}{2} \quad (27)$$

[0185] 未知函数 $\phi(x)$ 的数值解与解析解散点分布图如图2所示, 原函数 $F_0(x, t)$ 的数值解散点分布图如图3所示, 未知函数 $\phi(x)$ 的数值解与解析解的均方差 $\sigma_{\text{MSE}} = 2.5677 \times 10^{-8}$, 未知函数 $\phi(x)$ 的数值解与解析解的均方差最大值 $\sigma_{\text{MSE}, \text{max}} = 2.2287 \times 10^{-7}$, 神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}} = 4.5394 \times 10^{-10}$, 神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}} = 9.6574 \times 10^{-8}$ 。

[0186] 实施例3

[0187] 本实施例提供一种深度机器学习求解连续性积分方程的方法,包括以下步骤:

[0188] S1、对包含未知函数和积分项的连续性积分方程,使用牛顿-莱布尼茨公式将其中的积分项转化为被积函数的原函数表达式,将原积分方程转化为没有积分项的微分方程;

[0189] S2、给出被积函数原函数特定解的生成方法;

[0190] S3、对未知函数和被积函数原函数分别映射为神经网络函数,对未知函数和被积函数原函数的神经网络函数交替进行深度机器学习;

[0191] 完成深度机器学习的未知函数和被积函数原函数的神经网络函数分别为未知函数和被积函数原函数的数值解。

[0192] 在本实施例中,S1中,连续性积分方程为第二类线性Volterra方程,其表达式为:

$$[0193] \quad \phi(x) - \int_0^x \sin(x-t) \phi(t) dt = e^x \quad (28)$$

[0194] 其中,x为积分方程自变量,t为积分方程积分符号下的自变量, $\phi(x)$ 为未知函数;

[0195] 假设方程(23)中被积函数 $\sin(x-t) \phi(t)$ 是关于自变量t的连续函数,根据微积分学基本原理,存在被积函数 $\sin(x-t) \phi(t)$ 的原函数 $F(x,t)$,满足:

$$[0196] \quad F(x,t)' = \sin(x-t) \phi(t) \quad (29)$$

[0197] 根据牛顿-莱布尼兹公式有:

$$[0198] \quad \int_0^x \sin(x-t) \phi(t) dt = F(x,x) - F(x,0) \quad (30)$$

[0199] 将方程(30)代入方程(29)得到:

$$[0200] \quad \phi(x) - (F(x,x) - F(x,0)) = e^x \quad (31)$$

[0201] 从而将原积分方程转换为没有积分项的微分方程。

[0202] 在本实施例中,S2中,方程(29)的原函数为一组相差实数C的函数,称为原函数簇 $F_n(x,t)$;设 $F_1(x,t)$ 为原函数簇 $F_n(x,t)$ 中一个特定的原函数,则原函数簇 $F_n(x,t)$ 满足以下形式: $F_n(x,t) = F_1(x,t) + C_n$,其中, C_n 为任意的实数值, $n=0,1,2,\dots$;

[0203] 令 $C_n = -F_1(x,0)$,则有原函数簇 $F_n(x,t)$ 中确定的原函数 $F_0(x,t)$ 为:

$$[0204] \quad F_0(x,t) = F_1(x,t) - F_1(x,0) \quad (32)$$

[0205] 当 $t=0$ 时,有 $F_0(x,0)=0$,此时为原函数簇 $F_n(x,t)$ 在满足一定的约束条件下,找到了一个确定的原函数 $F_0(x,t)$,将方程(22)带入方程(31)得到:

$$[0206] \quad \phi(x) - F_0(x,x) = e^x \quad (33)$$

[0207] 此时,确定被积函数原函数特定解的约束条件为: $t=0, F_0(x,0)=0$ 。

[0208] 在本实施例中,S3中,将未知函数 $\phi(x)$ 和原函数 $F_0(x,t)$ 分别使用神经网络函数进行映射,同时分别构建相应的损失函数,通过交替迭代深度机器学习,收敛得到的未知函数 $\phi(x)$ 和原函数 $F_0(x,t)$ 的神经网络函数分别为未知函数 $\phi(x)$ 和原函数 $F_0(x,t)$ 的数值解。

[0209] 作为其中一种可能的实现方式,S3,包括如下步骤:

[0210] S30、基于物理信息指引的神经网络模型或其他神经网络模型,分别建立未知函数 $\phi(x)$ 和原函数 $F_0(x,t)$ 的神经网络函数;

[0211] S31、分别将未知函数 $\phi(x)$ 和原函数 $F_0(x,t)$ 的神经网络函数作为未知函数 $\phi(x)$ 和原函数 $F_0(x,t)$ 的试函数,带入微分方程得到控制方程损失函数;

[0212] S32、根据S31中的控制方程损失函数建立统一损失函数；

[0213] S33、根据S32中的统一损失函数，对未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的神经网络函数交替进行深度机器学习；

[0214] 完成深度机器学习的未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的神经网络函数分别为未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 的数值解。

[0215] 作为其中一种可能的实现方式，S30中，基于物理信息指引的神经网络模型，对未知函数 $\varphi(x)$ 和原函数 $F_0(x, t)$ 分别建立采用基本的全连接的神经网络函数 $N(\vec{x})$ ：

$$[0216] \quad N(\vec{x}) = f(\vec{x}, w, b, l, n) \quad (34)$$

[0217] 其中， $\vec{x}$ 为神经网络输入向量， $N(\vec{x})$ 为神经网络输出， w 为神经网络连接权重； b 为神经网络偏置项； l 为网络层数， $l=9$ ； n 为隐藏神经单元个数， $n=40$ 。

[0218] 需要说明的是，可以根据深度学习的需要，选择任意类型的神经网络函数来求解目标方程，本发明对神经网络函数的类型不做限定。

[0219] 作为其中一种可能的实现方式，S31中，对于控制方程损失函数，令 $N_\phi(\vec{x}) = \phi(x)$ ， $N_p(\vec{x}, \vec{t}) = F_0(x, t)$ ，其中， $N_\phi(\vec{x})$ 为未知函数 $\varphi(x)$ 的神经网络函数， $N_p(\vec{x}, \vec{t})$ 为原函数 $F_0(x, t)$ 的神经网络函数， $\vec{x}$ 表示积分方程自变量 x ， $\vec{t}$ 表示积分方程积分符号下的自变量 t ；

[0220] 用 $N_p'(\vec{x}, \vec{t})$ 表示 $N_p(\vec{x}, \vec{t})$ 关于 $\vec{t}$ 的导数/偏导数，即： $N_p'(\vec{x}, \vec{t}) = F_0'(x, t)$ ；

[0221] 将 $N_p'(\vec{x}, \vec{t})$ 和 $F_0'(x, t)$ 的等价关系代入方程 (33)，得到控制方程残差表示的控制方程损失函数 $Loss_\phi$ 为：

$$[0222] \quad Loss_\phi = \frac{1}{|I|} \sum_i [a(\vec{x}_i) N_\phi(\vec{x}_i) - \lambda N_p(\vec{x}_i, \vec{x}_i) - f(\vec{x}_i)]^2 \quad (35)$$

[0223] 将 $N_p'(\vec{x}, \vec{t})$ 和 $F_0'(x, t)$ 的等价关系代入方程 (29)，得到原函数 $F_0(x, t)$ 的导数/偏导数与未知函数 $\varphi(x)$ 形成的由残差表示的控制方程损失函数 $Loss_p$ 为：

$$[0224] \quad Loss_p = \frac{1}{|I|} \sum_i [N_p'(\vec{x}_i, \vec{t}_i) - K(\vec{x}_i, \vec{t}_i) M[N_\phi(\vec{t}_i)]]^2 \quad (36)$$

[0225] 其中， i 为机器学习需要的每个样本点，采用随机分布生成； I 为机器学习需要的样本点总数， $I=500$ 。

[0226] 作为其中一种可能的实现方式，S32中，将S31中的损失函数加权，得到神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $Loss_{\phi-all}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $Loss_{p-all}$ 为：

$$[0227] \quad Loss_{\phi-all} = P_\phi Loss_\phi \quad (37)$$

$$[0228] \quad Loss_{p-all} = P_p Loss_p \quad (38)$$

[0229] 其中， P_ϕ 为控制方程损失函数 $Loss_\phi$ 的权重， $P_\phi=1$ ； P_p 为控制方程损失函数 $Loss_p$ 的权重， $P_p=1$ 。

[0230] 作为其中一种可能的实现方式，S33中，神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $Loss_{\phi-all}$

和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 分别设定收敛准则;

[0231] 对神经网络函数 $N_\phi(\vec{x})$ 和 $N_p(\vec{x}, \vec{t})$ 交替进行深度机器学习,为根据方程(37)和(38)确定的加权损失函数,分别交替调整神经网络函数 $N_\phi(\vec{x})$ 和 $N_p(\vec{x}, \vec{t})$ 中的 w 和 b ,使得函数神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 分别满足收敛准则;

[0232] 当神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 分别满足收敛准则后,完成深度机器学习;

[0233] 完成深度机器学习的神经网络函数 $N_\phi(\vec{x})$ 和 $N_p(\vec{x}, \vec{t})$ 分别为未知函数 $\phi(x)$ 和原函数 $F_0(x, t)$ 的数值解;

[0234] 深度机器学习采用ADAM机器学习算法,激活函数选用tanh,神经网络初始值采用高斯随机分布,机器学习率从 $1e^{-4}$ 开始,随着学习次数增加,逐渐下降,训练到神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 均不再下降。

[0235] 作为其中一种可能的实现方式,S33中,神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 分别设定的收敛准则为:神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}}$ 和神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}}$ 分别小于等于设定值 ε 。

[0236] 未知函数 $\phi(x)$ 的解析解为:

[0237] $\phi(x) = 2e^x - x - 1$ (39)

[0238] 未知函数 $\phi(x)$ 的数值解与解析解散点分布图如图4所示,原函数 $F_0(x, t)$ 的数值解散点分布图如图5所示,未知函数 $\phi(x)$ 的数值解与解析解的均方差 $\sigma_{\text{MSE}} = 5.0023 \times 10^{-8}$,未知函数 $\phi(x)$ 的数值解与解析解的均方差最大值 $\sigma_{\text{MSE}, \max} = 2.5687 \times 10^{-6}$,神经网络函数 $N_\phi(\vec{x})$ 的损失函数 $\text{Loss}_{\phi\text{-all}} = 4.7322 \times 10^{-9}$,神经网络函数 $N_p(\vec{x}, \vec{t})$ 的损失函数 $\text{Loss}_{p\text{-all}} = 1.5876 \times 10^{-7}$ 。

[0239] 以上所述实施例仅表达了本发明的几种实施方式,其描述较为具体和详细,但不能因此而理解为本发明专利范围的限制。应当指出的是,对于本领域的普通技术人员来说,在不脱离本发明构思的前提下,还可以做出若干变形和改进,这些都属于本发明的保护范围。因此,本发明的保护范围应以所附权利要求为准。

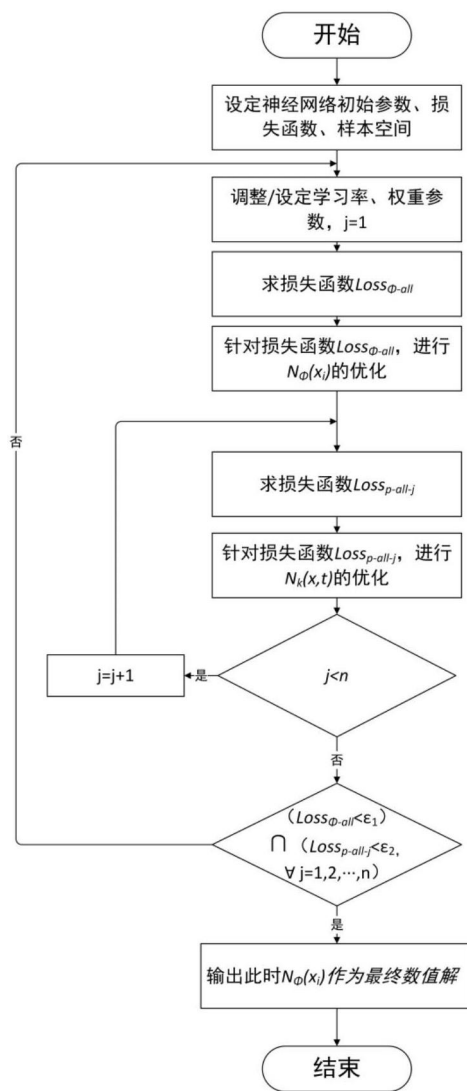


图1

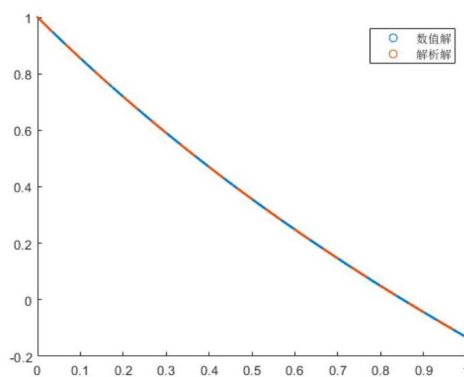


图2

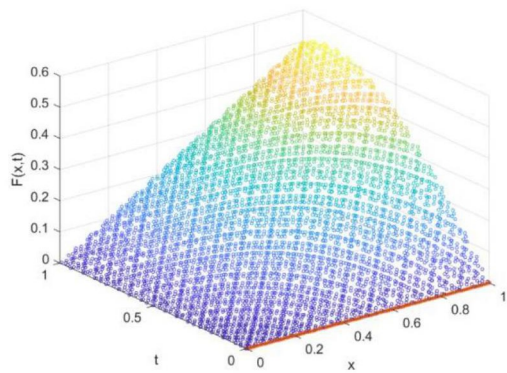


图3

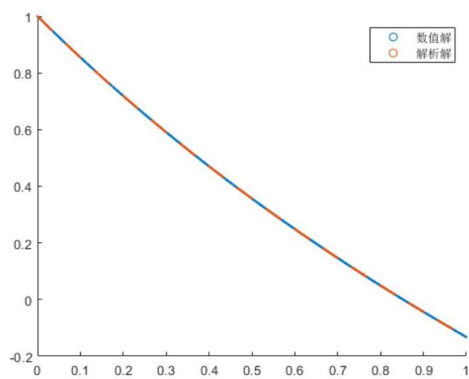


图4

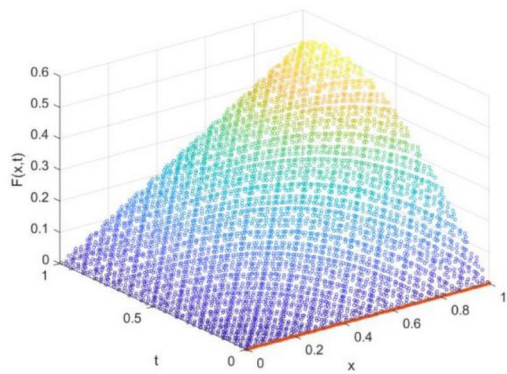


图5